Checkpoint-as-a-Service: Co-designing LLM Training and Storage for Efficient Checkpointing

Participant(s): Saam (SeyedMohammadHossein) Hosseini

Project proposal for the final course project

---

1. Project Goals

Large Language Model (LLM) training is expensive enough that checkpointing overhead matters. Whenever a training job saves model weights and optimizer state, GPUs can stall while data is serialized, compressed, routed, and written to durable storage. This project builds Checkpoint-as-a-Service (CPaaS), a checkpoint management layer that coordinates the training side and the storage side instead of treating them as independent systems.

- **Minimize checkpoint stall time** — reduce the visible time that training nodes wait for checkpoint I/O to finish.
- **Adaptively compress checkpoint data** — enable or disable client-side compression only when it helps overall throughput.
- **Balance upload load across storage nodes** — route traffic away from overloaded MinIO nodes to avoid hotspots.
- **Redistribute checkpoint chunks across training nodes** — move excess data between clients so one slow rank does not dominate checkpoint time.

Success criteria: measurable reduction in checkpoint completion time, higher aggregate upload throughput, lower skew across client ranks, and correct recovery after node stress or failure injection.

Why This Project Is Compelling

- **Real systems problem** — the bottleneck is not fake busywork; checkpointing can waste expensive GPU time and directly hurt training throughput.
- **Cross-layer systems design** — the project touches storage, networking, load balancing, control-plane logic, and distributed application behavior in one end-to-end system.
- **Clear before/after evaluation** — the project can be judged with concrete metrics rather than vague claims: time per checkpoint, throughput, straggler reduction, and recovery correctness.
- **Useful even if not every stretch goal lands** — the core system is valuable on its own, and each adaptive mechanism can be tested independently.

2. Specific Software and Hardware Components

2.1 Hardware

Component	Role	Specs
1x Load Balancer / Control Plane (sam-lb)	NGINX reverse proxy and central control plane node	e2-medium, 2 vCPU, 4 GB RAM

Component	Role	Specs
4x Storage Nodes (minio1-minio4)	Distributed MinIO object storage cluster	e2-standard-4, 4 vCPU, 15 GB RAM, 200 GB SSD each
4x Client / Compute Nodes (client1-client4)	Simulate distributed training ranks and checkpoint upload clients	n2-standard-2, 2 vCPU, 8 GB RAM, 250 GB SSD each

Deployment target: 9 GCP virtual machines in the same VPC and zone, using the cluster as a repeatable experimental testbed rather than a paper-only design.

2.2 Software

Component	Technology	Role
Distributed Object Storage	MinIO (S3-compatible, 4-node erasure-coded cluster)	Durable checkpoint storage backend
Load Balancer	NGINX	Routes S3 traffic and supports adaptive per-node weighting
Checkpoint Client	Python, PyTorch, Safetensors, boto3	Generates checkpoint files, compresses them, uploads them, and reports metrics
Checkpoint Coordinator	Python HTTP API on port 8765	Computes redistribution plans and tracks historical throughput
Chunk Transfer System	Custom TCP server/client on port 9876	Moves checkpoint chunks between training nodes
Adaptive LB Manager	Bash + NGINX config reload	Adjusts routing weights from observed storage-node load
Compression Control	ZSTD / S2 / GZIP	Adaptive client-side compression policy
Metrics Collection	Prometheus + SSH-based scripts	Collects CPU, memory, network, disk, and per-run metrics
Container Runtime	Docker	Packages services and improves deployment repeatability

3. Architectural Diagram

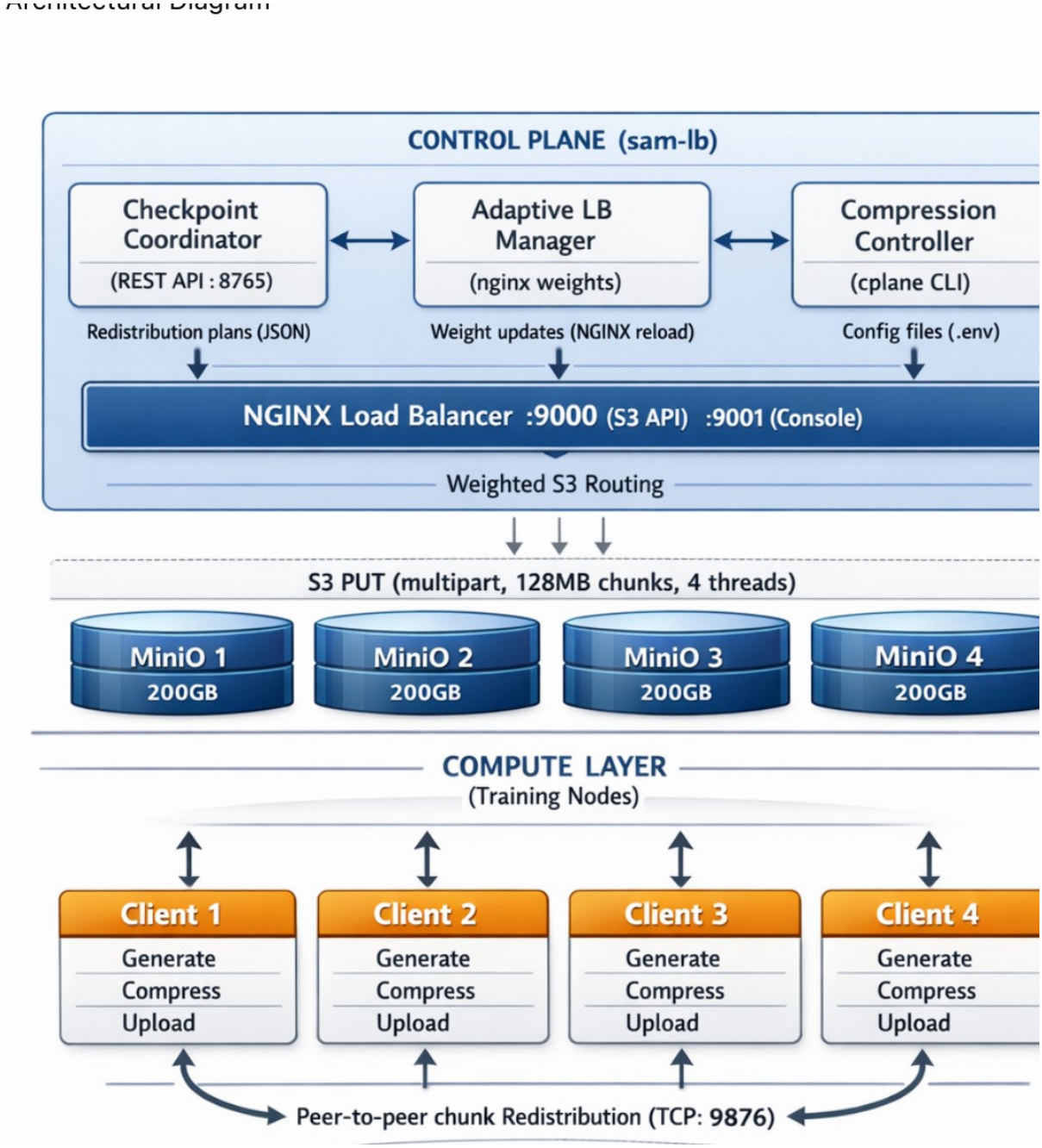

Figure 1. CPaaS architecture showing the control plane, storage layer, compute clients, and peer-to-peer redistribution path.

4. Interaction of Software and Hardware Components

4.1 Normal Checkpoint Path

Each client acts like a distributed training rank. It generates a checkpoint containing model weights and optimizer state, contacts the coordinator for a redistribution plan, optionally exchanges chunks with peers, applies the current compression policy, and uploads its final assigned data to MinIO through the NGINX load balancer using multipart S3 uploads.

4.2 Adaptive Compression

The control plane monitors storage-node CPU load. If storage is becoming the bottleneck, the controller can enable client-side compression so compute nodes spend extra CPU cycles to reduce network and storage pressure. If compression stops helping, the controller disables it.

4.3 Adaptive Load Balancing

The adaptive load-balancing manager periodically polls MinIO nodes, converts utilization into routing weights, rewrites the NGINX configuration, and reloads it. New multipart uploads are therefore steered away from hot nodes and toward healthier ones.

4.4 Peer-to-Peer Redistribution

When checkpoint sizes are uneven across clients, the coordinator computes a size-balancing redistribution plan. Clients split files into fixed-size chunks and transfer excess chunks over TCP. This directly targets the straggler problem: the slowest client should not dominate total checkpoint time if other clients are underutilized.

4.5 Metrics Feedback Loop

Metrics are collected from all nodes and fed back into the controller. This closes the loop between observation and decision-making: routing, compression, and redistribution are changed using actual cluster conditions instead of static assumptions.

5. Debugging, Training, and Testing Plan

- **Simulation mode** — the client can generate realistic synthetic checkpoint data without requiring a large production model, which makes debugging faster and cheaper.
- **Component-level tests** — TCP chunk transfer is validated with checksums and interrupted sends; coordinator APIs are unit-tested; compression is checked for correctness; NGINX reconfiguration is verified through effective routing changes.
- **End-to-end experiments** — each run has baseline, stress, and adaptive phases so the benefit of each mechanism can be isolated instead of hand-waved.
- **Fault injection** — selected MinIO nodes can be stressed or temporarily disrupted to verify that the control plane reacts correctly and that saved checkpoints remain readable.
- **Training/test mechanism** — the initial workload uses PyTorch-based distributed checkpoint clients with synthetic state; a stretch goal is to integrate the same ideas into a real training stack such as DeepSpeed.

6. Why This Meets the Four Cloud Technologies Requirement

This project clearly uses more than four distinct cloud-relevant technologies. The strongest four are below, with two additional technologies included for completeness.

Technology	How it is used in this project
Virtual machines / cloud compute	Nine GCP virtual machines host the control plane, storage servers, and client nodes.
Containers	Docker is used to package and deploy services consistently across machines.

Technology	How it is used in this project
Storage service / object storage API	MinIO provides S3-compatible distributed object storage for durable checkpoint persistence.
RPC / API interfaces	The coordinator exposes an HTTP/JSON API, while checkpoint persistence uses the S3 API.
Networking / load balancing	A GCP VPC connects all nodes, and NGINX performs adaptive weighted routing for storage traffic.
Message encoding / transport	JSON messages coordinate control decisions and a custom TCP protocol transfers checkpoint chunks between clients.

This is not just one library wrapped in a script. It is a distributed service with storage, control-plane logic, routing, data transfer, and repeatable cloud deployment.

7. Scope, Deliverables, and Fallback Plan

- **Core deliverables by the end of the class** — a working MinIO cluster, an NGINX-based adaptive routing layer, a checkpoint coordinator, client-side compression control, peer-to-peer chunk redistribution, and a reproducible evaluation suite.
- **Stretch goals** — integration with a real training framework such as DeepSpeed, local-SSD staging, and overlap techniques that further hide checkpoint overhead.
- **Why this is within scope** — the system is modular. Each major mechanism can be implemented and tested independently, so progress is still meaningful even if every stretch goal is not completed.
- **Why this is ambitious enough** — the project still requires building and evaluating a full distributed pipeline rather than just tuning one script or running a single benchmark.

8. Expected Outcome

The expected result is a proposal and prototype that are both ambitious and realistic: the class project delivers a fully working adaptive checkpointing pipeline on cloud VMs, demonstrates measurable performance gains under hotspot and imbalance scenarios, and leaves room for a stronger research extension after the course.

End of proposal